HOW-TO

CSS page break print: force, avoid and fix page breaks

Updated 1 Dec 2026

KEY TAKEAWAYS

- `break-before: page` forces a break; `break-inside: avoid` asks the browser not to split a box.
- The old `page-break-*` names are aliases of the `break-*` properties; MDN marks them deprecated.
- `avoid` on `break-before` and `break-after` is recognized but has no effect in Firefox and Safari, per MDN.
- `break-inside: avoid` gives way when there is no other place to break: the spec then drops the avoid rules and, as a last resort, lets the browser break anywhere.
- Page-margin boxes for page numbers appear only in Chromium-based browsers in MDN's table today.

CONTENTS

1 Force a CSS page break for print 2
2 Avoiding a break 3
3 Why break-inside: avoid is not working 4
4 Tables across pages 4
5 Page numbers 5
6 The other way: fixed sheets 6
7 What to remember 6

To control a CSS page break in print, use `break-before: page` to force a new page and `break-inside: avoid` to ask the browser not to split a box. The rules work inside `@media print` or outside it. The old `page-break-before`, `page-break-after` and `page-break-inside` names still work, because browsers treat them as aliases of the new ones.

The short version:

```
@media print {
  h2.chapter { break-before: page; }         /* every chapter starts a page */
  h2, h3 { break-after: avoid; }             /* keep a heading with what follows
*/
  figure, tr, .card { break-inside: avoid; } /* do not split these */
  p { orphans: 3; widows: 3; }               /* no lone lines at page edges */
}
```

MDN's printing guide says styles in `@media print` are used only for printed content, and that includes saving as PDF. That makes it the safe place for rules you don't want on screen. The rest of this page goes through each line and where it stops working.

1. Force a CSS page break for print

`break-before` and `break-after` say what should happen before and after a box. The values that matter for pages:

- `page` forces a page break at that point.
- `left`, `right`, `recto` and `verso` force one or two breaks so the next page is a left, right, recto or verso page. These are for books.
- `avoid` and `avoid-page` ask the browser not to break there.
- `auto` is the default: a break is allowed, not forced.

For a document with chapters, put the break on the element that starts each one:

```
.chapter { break-before: page; }
```

If you'd rather think about where a section ends, `break-after: page` on its last element does the same job.

MDN says both properties apply to block-level elements. The CSS Fragmentation spec adds grid items, flex items, table row groups and table rows, and says browsers must not apply them to absolutely positioned boxes. MDN lists the `page` value for `break-before` and `break-after` from Chrome 50, Edge 79, Firefox 65 and Safari 10.

Forced values win. Every possible break point is governed by three properties: the `break-after` of the box before it, the `break-before` of the box after it, and the `break-inside` of the box that contains both. When one of them forces a break, MDN says it takes precedence, and when more than one does, `break-before` beats `break-after`, which beats `break-inside`.

Sizing the sheet itself is a separate job, covered in css page size a4.

The old names

`page-break-before: always` is the same as `break-before: page`. MDN maps `auto`, `left`, `right` and `avoid` one to one, and `always` to `page`. It marks the old properties deprecated and replaced by `break-*`. Keep them only where a stylesheet already has them.

2. Avoiding a break

`break-inside: avoid` on a box asks the browser to keep it in one piece. MDN's example puts it on a `figure` so the image and its caption stay together. Cards, list items, signature blocks and table rows are the usual candidates.

`break-after: avoid` on a heading asks for no break between it and what follows, which is how you'd stop a heading sitting alone at the bottom of a page. But MDN's tables show `avoid` on `break-before` and `break-after` as unsupported in Firefox and Safari: the value is recognized and has no effect. It works in Chrome 50 and Edge 79 onwards. `break-inside: avoid`, on the other hand, is listed from Chrome 50, Edge 79, Firefox 65 and Safari 10. So wrap the heading and its first paragraph in one element and put `break-inside: avoid` on the wrapper.

Orphans and widows

`orphans` is the minimum number of lines of a paragraph left at the bottom of a page; `widows` is the minimum carried to the top of the next. Both default to 2 and take a positive integer. MDN shows support in Chrome 25, Edge 12 and Safari 1.3 and none in Firefox, where its example behaves as if the value were 1. A value of 3 avoids two-line scraps at page edges in the browsers that support it.

3. Why break-inside: avoid is not working

When `break-inside: avoid` seems to do nothing, one of these is usually behind it. The first two follow from the rules in the CSS Fragmentation spec; the rest are stated by MDN or the spec directly.

1. **There's nowhere else to break.** The spec gives four rules that decide where an unforced break may fall: the avoid values of `break-before` and `break-after`, an ancestor's `break-inside: avoid`, and orphans and widows. If they leave too few break points to keep content from overflowing the page, the spec drops the orphans and widows rule first, then the avoid rules. If that still isn't enough, the browser may break anywhere to avoid losing content. A card 30 cm tall can't be kept whole on a 29.7 cm page.
2. **The avoid is set too high.** Under those rules, an ancestor with `break-inside: avoid` forbids breaks between lines inside it, and between its children where they don't set a break themselves. Put it on `main` or `body` and nearly every break point in the document is forbidden. Once the content doesn't fit, the spec has the browser drop the avoid rules and pick other points. It may weigh the avoids when it does, but the spec doesn't say how, so the result can look arbitrary.
3. **A forced break sits at the same point.** Forced beats avoid, as above.
4. **There's no box, or the wrong kind of box.** MDN says the property is ignored when there is no generated box, and that it applies to block-level elements. The spec excludes inline-level boxes and absolutely positioned boxes, so a plain `span` or a `position: absolute` element won't obey it.
5. **The value isn't supported where you print.** `avoid` on `break-before` and `break-after` has no effect in Firefox and Safari, and `orphans` and `widows` are ignored in Firefox.

The spec has one more last resort: content that can't be broken, such as an image, may be sliced across two pages. If a large image gets cut, make it small enough to fit on one page.

---

4. Tables across pages

Tables are where print page breaks go wrong most visibly: a row split in half, or a header that only appears on the first page.

For rows, the spec says `break-before` and `break-after` apply to table rows and row groups. `break-inside` applies to every element except inline-level boxes, internal ruby boxes, table columns and column groups, and absolutely positioned boxes. So this rule is valid:

```
@media print {
  tr { break-inside: avoid; }
}
```

MDN's compatibility data has no rows specific to tables, so check the result in the browsers you print from.

For headers, use `thead`. CSS 2.1 maps `thead` to `display: table-header-group` and says print user agents may repeat header rows on each page a table spans; the same goes for `tfoot` and footer rows. "May" means a browser is allowed to, not required to. Treat a repeated header as something to verify.

5. Page numbers

CSS has an answer here, and most browsers support only part of it. The CSS Paged Media draft defines a `page` counter that is created automatically and goes up by one on every page, and a `pages` counter that holds the total. You show them in a page-margin box:

```
@page {
  margin: 20mm;
  @bottom-center { content: counter(page) " / " counter(pages); }
}
```

MDN lists the page-margin boxes, marked experimental, in Chrome 131, Edge 131 and Opera 116, and as unsupported in Firefox and Safari. If readers print from those, write the number into the HTML yourself, one footer per page.

When a script makes the PDF, the tool may number pages for you. Playwright's `page.pdf()` has a `displayHeaderFooter` option, and its `headerTemplate` and `footerTemplate` fill in elements with the classes `pageNumber` and `totalPages`.

6. The other way: fixed sheets

Everything above lets the browser choose where a page ends and nudges it with hints. The opposite approach is to choose yourself. You split the content into fixed-size sheets, one element per printed page, so every break is one you placed. No hint can fail, because the browser never has to pick a break point. The price is that you have to keep each sheet short enough to fit.

papyrus, a toolkit by n-uf for documents rendered as HTML and exported as PDF, works this way. Its page contract says each element with the class `pdf-page` is one sheet on screen and one page in the PDF. In print, `sheetCss()` gives each of those elements `break-after: page` and `break-inside: avoid-page`, along with the old `page-break-*` names. The exporter then counts the page elements, the PDF's pages and the expected `pageCount`, and won't write the PDF when they disagree. A sheet that spilled onto an extra page is caught before the file is saved.

For a table of contents, `numberToc()` numbers the entries: it returns a map from entry id to `"1."`, `"2."`, `"A."` for an appendix, or `null` for an entry it skips. In `@n-uf/papyrus-react`, a `TOCItem` takes the page it points to as a prop. The exporter prints with headless Chromium through Playwright; papyrus and n-uf are not affiliated with Microsoft's Playwright project or with the browser makers named here.

papyrus is young. Its versions are dates (`YY.M.R`), not semantic versions, so any release may break things. Pin an exact version and read the CHANGELOG. The html to pdf a4 page has the details.

---

7. What to remember

- `break-before: page` forces a break. `break-inside: avoid` only asks, and the browser may have to say no.
- Use the `break-*` names. The `page-break-*` ones are deprecated aliases.
- `avoid` on `break-before` and `break-after` does nothing in Firefox and Safari. Wrap the elements and use `break-inside` instead.
- Check tables in the browsers you target. Repeating a `thead` is allowed, not required.
- With fixed sheets you place every break, and a page-count check catches an overflow.

Questions people ask

How do I force a page break in CSS?

Set `break-before: page` on the element that should start a new page, or `break-after: page` on the one before it. MDN says `break-before: page` forces a page break right before the box, and that the property applies to block-level elements. The older `page-break-before: always` is an alias.

Why is break-inside: avoid not working?

Most often the box doesn't fit, so the browser has to break it somewhere. The CSS Fragmentation spec says that when its rules leave too few break points, the avoid rules are dropped, and as a last resort the browser may break anywhere to avoid losing content. Other causes are a forced break at the same point, or an element that isn't a block-level box.

Does page-break-after still work?

Yes. MDN says browsers should treat the `page-break-*` properties as aliases of `break-*`, with `always` mapping to `page`. MDN also marks them deprecated and replaced by the `break-*` properties, so write `break-after` in new code.

How do I repeat a table header on every printed page?

Put the header row in a `thead`. CSS 2.1 says print user agents may repeat header rows on each page a table spans. The word is may, so look at your target browser's print preview before you rely on it.

How do I add page numbers when printing from CSS?

Use a page-margin box and the `page` counter, for example `@page { @bottom-center { content: counter(page); } }`. MDN's table lists the margin boxes in Chrome 131, Edge 131 and Opera 116, and not in Firefox or Safari, so if you need numbers everywhere, put them in the HTML.

Sources

1. MDN: break-before ↗ checked 11 Nov 2026
2. MDN: break-after ↗ checked 11 Nov 2026
3. MDN: break-inside ↗ checked 11 Nov 2026
4. MDN: page-break-before ↗ checked 11 Nov 2026
5. MDN: orphans ↗ checked 11 Nov 2026
6. MDN: widows ↗ checked 11 Nov 2026
7. MDN: @page at-rule ↗ checked 11 Nov 2026
8. MDN: Printing ↗ checked 11 Nov 2026
9. W3C: CSS Fragmentation Module Level 3 (Candidate Recommendation, 4 December 2018) ↗ checked 11 Nov 2026
10. W3C: CSS Paged Media Module Level 3 (Working Draft, 14 September 2023) ↗ checked 11 Nov 2026
11. W3C: CSS 2.1, Tables ↗ checked 11 Nov 2026
12. Playwright: Page, page.pdf() ↗ checked 11 Nov 2026
13. papyrus README (@n-uf/papyrus v26.11.1) ↗ checked 11 Nov 2026
14. papyrus source: src/core/sheet.js (v26.11.1) ↗ checked 11 Nov 2026
15. papyrus-react README (@n-uf/papyrus-react v26.11.1) ↗ checked 11 Nov 2026